

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Департамент программной
инженерии ФКН НИУ ВШЭ
Научный руководитель,
кандидат технических наук,
доцент департамента
программной инженерии
факультета компьютерных наук

_____ / Н.С. Белова /
«__» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.02-06 81 01–1 ЛУ

Исполнители:
студент группы БПИ233

_____ / Б.А. Багавиев /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН
RU.17701729.02-06 81 01–1 ЛУ

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Пояснительная записка

RU.17701729.02-06 81 01–1

Листов 14

Изн.№ подл.	Подп. и дата	Взам. инв.№	Изн.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ «Пояснительная записка» разработан в соответствии с ГОСТ 19.404–79 и содержит обоснование технических решений, принятых при разработке программного изделия «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием» (BunnyMQ).

Документ описывает назначение и область применения системы, алгоритмы функционирования сегментированного журнала, конечных автоматов Raft, координаторов и API, а также обоснование выбора технологического стека и ожидаемые показатели производительности.

Документ составлен в соответствии с требованиями ГОСТ 19.105–78 и ГОСТ 19.404–79.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	2
2.1. Назначение программы	2
2.2. Область применения	2
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	3
3.1. Постановка задачи на разработку	3
3.1.1. Описание задачи	3
3.1.2. Описание применяемых методов	3
3.1.3. Ограничения версии v1	4
3.2. Описание алгоритма и функционирования программы	4
3.2.1. Архитектура узла	4
3.2.2. Подсистема хранения	5
3.2.3. MetadataFSM	5
3.2.4. PartitionFSM	5
3.2.5. Координаторы	6
3.2.6. gRPC API	6
3.3. Описание и обоснование выбора метода организации входных и выходных данных	6
3.3.1. Входные данные	6
3.3.2. Выходные данные	6
3.4. Описание и обоснование выбора состава технических и программных средств	7
3.4.1. Go 1.25	7
3.4.2. dragonboat v4	7
3.4.3. gRPC + Protocol Buffers	7
3.4.4. Prometheus	7
3.4.5. go.uber.org/zap	7
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	8
4.1. Показатели производительности	8
4.2. Показатели надёжности	8
4.3. Показатели ресурсоёмкости	8
4.4. Экономические показатели	9
5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	10

1. ВВЕДЕНИЕ

Наименование программы – «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием» (в тексте – BunnyMQ).

Условное обозначение по ГОСТ 19.103-77: **RU.17701729.02-06**.

Основание разработки – техническое задание с тем же наименованием (обозначение: RU.17701729.02-06 ТЗ 01–1), принятое в 2026 г. на факультете компьютерных наук НИУ ВШЭ.

Исполнитель – студент программы «Программная инженерия», группа БПИ233, Б.А. Багавиев.

В состав изделия входят серверный процесс `bunnymq`, утилита командной строки `bunnymq-cli` и публичная библиотека `pkg/client`. Настоящая записка охватывает серверную часть: подсистему хранения, FSM Raft, координаторы и gRPC API.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

BunnyMQ – распределённый брокер сообщений, по архитектуре близкий к Apache Kafka: данные организованы по топикам и партициям, потребление идёт по смещениям, однако вместо ISR-механизма используется Raft-репликация на уровне каждой партиции через dragonboat v4.

Три компонента системы:

- `bunnymq` – серверный процесс; принимает gRPC-запросы и пишет данные через Raft FSM в сегментированный журнал на диске;
- `bunnymq-cli` – административный CLI для управления кластером и топиками;
- `pkg/client` – Go-библиотека с автоматическим поиском лидера и поддержкой групп консьюмеров.

2.2. Область применения

Система рассчитана на применение в backend-инфраструктуре, где нужно разделить компоненты через очередь сообщений, сохранить порядок внутри партиции и масштабировать пропускную способность горизонтально. Типичные сценарии – сбор телеметрии, конвейеры обработки событий, очереди задач.

Целевые пользователи – backend-разработчики, встраивающие `pkg/client` в прикладной код, и DevOps-инженеры, развёртывающие кластер

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку

3.1.1. Описание задачи

Требуется реализовать распределённый брокер сообщений. Минимальный функциональный набор: управление топиками (создание, удаление, изменение числа партиций и политики хранения), публикация пакетов с выбором гарантии `acks=0` или `acks=all`, чтение по смещению с долгим опросом при отсутствии данных, управление группами консьюмеров с диапазоным назначением партиций.

Нефункциональные требования: корректное восстановление после обрыва записи (`torn write`), периодическая очистка данных по политике хранения, метрики Prometheus, структурированное логирование.

3.1.2. Описание применяемых методов

Сегментированный журнал. Данные каждой партиции хранятся в виде последовательности сегментов. Сегмент – три файла с одинаковым именем (базовое смещение первого пакета, дополненное нулями до 20 знаков): `.log` с самими пакетами, `.index` с индексом смещений и `.timeindex` с временным индексом. Запись идёт только в последний активный сегмент с флагами `O_APPEND|O_WRONLY`. Индексные файлы хранятся в mmap-отображениях; заполнение разрежено – индекс обновляется не на каждый пакет, а по достижении порогового числа байт. Поиск по смещению – бинарный по индексу, затем линейный по `.log`.

Формат пакета. Формат на диске совпадает с форматом на проводе, поэтому при передаче данных консьюмеру перекодирование не нужно. Пакет начинается с 38-байтного заголовка – базовое смещение, длина, число записей, контрольная сумма CRC-32C, атрибуты и две временные метки; записи внутри кодируются через `varint`.

Raft-репликация. Используется `dragonboat v4` в схеме `multi-raft`: один `shard` на партицию плюс один `shard` для метаданных кластера. `acks=0` реализован через асинхронный `Propose`, `acks=all` – через блокирующий `SyncPropose`.

Детерминизм FSM. `MetadataFSM` и `PartitionFSM` не обращаются к системному времени и не генерируют случайных значений самостоятельно – все нефиксированные

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

данные (временные метки, идентификаторы участников групп) передаются в теле команды от координатора, что является обязательным условием корректного воспроизведения Raft-журнала на репликах.

Долгий опрос. После каждого успешного Append Storage переключает внутренний канал уведомления newDataCh. DataCoordinator при обработке Fetch фиксирует снимок канала до первой попытки чтения. Если данных нет – ждёт закрытия канала или истечения max_wait_ms. Закрытие канала сигнализирует о появлении нового пакета.

Range Assignment. При ребалансировке группы партии и участники сортируются по идентификаторам, затем нарезаются на последовательные диапазоны. Первые $P \bmod M$ участников получают на одну партию больше, остальные – базовый размер. Алгоритм совпадает с реализацией в Apache Kafka.

Torn write recovery. При каждом Append FSM записывает применённый индекс Raft в файл applied.idx, а при запуске Open сравнивает реальное состояние журнала с контрольной точкой и отбрасывает хвост, если обнаруживает расхождение.

3.1.3. Ограничения версии v1

acks=1 намеренно не реализован – при Raft это не нужно, есть acks=0 для скорости и acks=all для надёжности. Снимки dragonboat отключены: FSM восстанавливается воспроизведением полного журнала. Состав кластера задаётся в конфигурации при запуске и не меняется в рантайме. Транзакционная запись не поддерживается.

3.2. Описание алгоритма и функционирования программы

3.2.1. Архитектура узла

Каждый узел – один процесс bunnymq с несколькими слоями:

```
cmd/bunnymq/
├─ main.go          -- инициализация и graceful shutdown
internal/
├─ storage/         -- сегментированный журнал
├─ partition/       -- PartitionFSM (IOOnDiskStateMachine)
├─ metadata/        -- MetadataFSM (IStateMachine)
├─ raft/            -- обёртка NodeHost
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата



Схема взаимодействия компонентов приведена в Приложении 1.

3.2.2. Подсистема хранения

Storage ведёт упорядоченный набор сегментов. При Append проставляет `base_offset` в заголовок пакета, передаёт пакет активному сегменту, при необходимости запечатывает его и создаёт новый. После записи переключает `newDataCh`.

Read находит нужный сегмент бинарным поиском по набору, локализует позицию через `.index`-файл и читает пакеты линейно до исчерпания `maxBytes`.

Фоновая горутина раз в минуту проверяет сегменты на соответствие политике хранения по времени и объёму, но активный сегмент не удаляется

3.2.3. MetadataFSM

Хранит состояние кластера в памяти (топики, партиции, группы консьюмеров, узлы), снимок – JSON-сериализация всего состояния. Чтение идёт через прямой `Lookup`, запись – только через `Raft`, а команды упакованы в JSON-конверт `MetadataCommand`.

3.2.4. PartitionFSM

Управляет одной репликой одной партиции, единственная команда – `AppendBatch`: вызывает `Storage.Append`, синхронизирует данные на диск, обновляет `applied.idx`. При открытии сравнивает реальное состояние журнала с контрольной точкой и усекает хвост при расхождении.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.5. Координаторы

ClusterCoordinator запускает shards dragonboat и отслеживает смену лидера – при смене обновляет соответствующие поля в MetadataFSM.

DataCoordinator маршрутизирует запросы Data API: если текущий узел – лидер нужной партии, обрабатывает запрос локально; иначе возвращает NotLeader с адресом реального лидера. Для Fetch реализует долгий опрос.

GroupCoordinator управляет группами консьюмеров через MetadataFSM: JoinGroup генерирует member_id на стороне сервера и добавляет участника, а фоновая горутина каждые 5 секунд исключает участников с истёкшим session timeout.

3.2.6. gRPC API

ManagementService (9091) и DataService (9090) – два независимых gRPC-сервера. Перед каждым методом auth interceptor проверяет заголовок bunnymq-auth-token при наличии токенов в конфигурации. Метрики Prometheus доступны на порту 9092.

3.3. Описание и обоснование выбора метода организации входных и выходных данных

3.3.1. Входные данные

Все клиентские взаимодействия – через gRPC поверх HTTP/2, а схемы запросов описаны .proto-файлами пакета bunnymq.v1. Пакеты сообщений передаются в сыром байтовом формате, совпадающем с форматом хранения.

Конфигурация узла задаётся YAML-файлом (флаг --config) и загружается при старте однократно; SIGTERM и SIGINT инициируют graceful shutdown.

3.3.2. Выходные данные

gRPC-ответы в формате Protobuf. При Fetch пакеты передаются в поле records bytes без перекодирования – та же байтовая последовательность, что хранится на диске. Метрики – text/plain в формате OpenMetrics по HTTP. Логи – JSON на stdout.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.4. Описание и обоснование выбора состава технических и программных средств

3.4.1. Go 1.25

Горутинная модель позволяет обслуживать сотни одновременных gRPC-соединений без потока на каждое, сборщик мусора с малыми паузами не вносит неконтролируемых задержек на горячем пути записи, а статическая компиляция без внешних зависимостей упрощает развёртывание.

3.4.2. dragonboat v4

Единственная Go-реализация Raft с поддержкой multi-raft из коробки – это принципиально важно: сбой одного shard не блокирует другие. Интерфейс `IOOnDiskStateMachine` даёт `PartitionFSM` возможность работать с внешним хранилищем без привязки к snapshot-механизму dragonboat. Альтернативы (etcd/raft, hashicorp/raft) multi-raft не поддерживают и требуют значительно большего объёма интеграционного кода.

3.4.3. gRPC + Protocol Buffers

Строгая типизация proto-схем исключает несогласованность между клиентом и сервером. Бинарный формат Protobuf компактнее JSON при передаче пакетов сообщений, а HTTP/2 снижает накладные расходы при параллельных запросах. Совпадение формата хранения с wire-форматом позволяет отдавать данные консьюмеру без промежуточной десериализации.

3.4.4. Prometheus

Pull-модель не требует изменений в сервисе при добавлении scrapers. dragonboat экспортирует собственные метрики в глобальный реестр Prometheus при включении `EnableMetrics`

3.4.5. go.uber.org/zap

Собирает JSON-строки без рефлексии – на горячем пути записи это важно, прочие логгеры добавляли бы измеримые накладные расходы. Уровень логирования задаётся в конфигурационном файле

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Показатели производительности

Оценки получены на основании бенчмарков дисковой записи (`benchmarks/disk_test.go`) и характеристик `dragonboat v4`.

Таблица 4.1 — Ожидаемые показатели производительности BunnyMQ

Показатель	Значение	Примечание
Запись (<code>acks=all</code> , 1 партиция)	≥ 50 МБ/с	Ограничена диском и задержкой Raft
Запись (<code>acks=0</code> , 1 партиция)	≥ 200 МБ/с	Ограничена только диском
Задержка Produce (<code>acks=all</code> , p99)	≤ 50 мс	Два RTT Raft + <code>fsync</code>
Задержка Produce (<code>acks=0</code> , p99)	≤ 5 мс	Без ожидания кворума
Задержка Fetch (есть данные, p99)	≤ 10 мс	Чтение с диска без репликации
Уведомление при long-poll	≤ 5 мс	Через <code>newDataCh</code> после Append
Минимальный размер кластера	3 узла	Raft-кворум: 2 из 3
Максимальный пакет	1 МиБ	Ограничение gRPC по умолчанию

4.2. Показатели надёжности

Отказ одного узла из трёх не нарушает доступность – кворум сохраняется. `Topic write` исправляется автоматически при перезапуске за счёт сравнения `applied.idx` с реальным состоянием журнала. Минимальная единица восстановления – целый пакет; частично записанные пакеты отбрасываются.

4.3. Показатели ресурсоёмкости

Таблица 4.2 — Минимальные требования к ресурсам узла

Ресурс	Требование
CPU	2 ядра x86-64; 4 рекомендовано при нагрузке более 10 000 RPS
RAM	512 МБ без учёта mmap-индексов; индексы вытесняются ОС при нехватке памяти
Диск	SSD рекомендован; объём – <code>retention_bytes</code> × число партиций на узле плюс журналы Raft
Сеть	TCP-связность между всеми узлами; задержка менее 100 мс

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.4. Экономические показатели

Разработка выполнена в рамках учебного проекта. Все используемые библиотеки распространяются под открытыми лицензиями (Go – BSD, dragonboat – Apache 2.0, gRPC-Go – Apache 2.0, Prometheus client_golang – Apache 2.0, zap – MIT). Для запуска трёх узлов достаточно трёх виртуальных машин стандартной конфигурации, стоимостью порядка 1 500–3 000 руб./мес. каждая в публичных облаках.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.404–79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. — М.: Стандартиформ, 2010.
2. ГОСТ 19.105–78. Единая система программной документации. Общие требования к программным документам. — М.: Стандартиформ, 2010.
3. ГОСТ 19.106–78. Единая система программной документации. Требования к программным документам, выполненным печатным способом. — М.: Стандартиформ, 2010.
4. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М.: Стандартиформ, 2010.
5. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC „14, 2014. [Электронный ресурс]. — URL: <https://raft.github.io/raft.pdf> (дата обращения: 26.04.2026).
6. The Go Programming Language Specification. Версия 1.25. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 26.04.2026).
7. dragonboat: A Multi-Group Raft library in Go. Документация dragonboat v4. [Электронный ресурс]. — URL: <https://github.com/lni/dragonboat> (дата обращения: 26.04.2026).
8. Apache Kafka Documentation. Версия 3.7. [Электронный ресурс]. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 26.04.2026).
9. Kreps J., Narkhede N., Rao J. Kafka: a Distributed Messaging System for Log Processing. NetDB „11, 2011. [Электронный ресурс]. — URL: <https://notes.stephenholiday.com/Kafka.pdf> (дата обращения: 26.04.2026).
10. gRPC Documentation. [Электронный ресурс]. — URL: <https://grpc.io/docs/> (дата обращения: 26.04.2026).
11. Protocol Buffers Documentation. Версия proto3. [Электронный ресурс]. — URL: <https://protobuf.dev/> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

12. Prometheus Documentation. [Электронный ресурс]. — URL: <https://prometheus.io/docs/> (дата обращения: 26.04.2026).
13. Uber go/zip. Blazing fast, structured, leveled logging in Go. [Электронный ресурс]. — URL: <https://github.com/uber-go/zip> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1**ОПИСАНИЕ ПОТОКОВ ДАННЫХ УЗЛА**

Входящий gRPC-запрос от клиента проходит через auth interceptor, после чего попадает в ManagementService (порт 9091) или DataService (порт 9090) в зависимости от типа операции.

ManagementService передаёт команды в ClusterCoordinator, который пишет их в MetadataFSM через raft.Host (shard 0).

DataService передаёт запросы в DataCoordinator. При публикации DataCoordinator проверяет, является ли текущий узел лидером нужной партиции, и либо обрабатывает запрос через PartitionFSM, либо возвращает адрес лидера. PartitionFSM пишет данные в Storage; после каждого успешного Append Storage закрывает newDataCh, что разблокирует ожидающие Fetch-запросы.

GroupCoordinator обрабатывает операции групп консьюмеров (JoinGroup, Heartbeat, LeaveGroup, CommitOffset) через MetadataFSM.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ФОРМАТ ПАКЕТА ДАННЫХ

Таблица П2.1 — Заголовок пакета (38 байт, big-endian)

Смещение	Длина	Тип	Поле
0	8	int64	base_offset – базовое смещение первой записи; заполняется Storage
8	4	int32	batch_length – полная длина пакета в байтах
12	4	int32	record_count – число записей в пакете
16	4	uint32	crc32 – CRC-32C над bytes[38..batch_length)
20	2	int16	attributes – зарезервировано (0 в v1)
22	8	int64	base_timestamp – временная метка первой записи, мс от Unix-эпохи
30	8	int64	max_timestamp – временная метка последней записи, мс
38	var	[]byte	records – varint-кодированные записи

Таблица П2.2 — Запись индекса смещений (.index, 8 байт)

Байты	Тип	Поле
0–3	int32	relative_offset = base_offset – segment.base_offset
4–7	int32	position – смещение пакета в .log-файле

Таблица П2.3 — Запись временного индекса (.timeindex, 12 байт)

Байты	Тип	Поле
0–7	int64	timestamp_ms = batch.max_timestamp
8–11	int32	relative_offset

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]